

(3) 边缘计算。将过去统一到中央进行整体计算,分散到数据产生、存储和查询端,数据产生既符合通用数据模型。同时,也给与实时数据模型反馈,让客户端传送数据的同时马上进行反馈,而不需要所有事件都要到中央端处理之后再行下发。

但 IOTA 架构需要对实时数据流和历史数据进行分别处理,增加了系统复杂度和开发难度;数据流处理和批处理的结果可能存在一定的不一致性,需要通过其他机制进行数据同步和数据一致性的保证;对于高并发、高吞吐量的场景,需要部署大量的计算节点和存储节点,需要更高的硬件成本和管理成本;需要设计和实现适合自身业务场景的数据处理流程和数据访问接口,对于非专业人员来说可能存在一定的难度。

19.3 大数据处理系统开发

大数据处理系统开发指的是通过大数据技术构建一个大数据处理系统的过程。

定义了大数据处理系统的应用需求、选择了架构,并进行适量的设计后,即澄清了“什么”的问题,接下来就进入构建阶段,即“如何”实现这些设计。构建阶段包括一系列的选择、编码、内容创建、集成、重构以及测试活动,以构建出可以部署并交付给最终用户使用的大数据处理系统应用。选择什么样的实现技术,是大数据处理系统成功的一个关键因素,最基本的原则是从内容、表示、功能分开考虑,而且需要考虑架构中分布式及与其他系统之间的交互。因此,需要了解各种技术的特点以及这些技术适用于什么样的架构,以及细粒度增量的部署。

大数据处理应用的实现技术与传统软件实现技术相比,最大的特性来自强大的数据管理能力,因此,本节从数据存储、数据管理、数据处理、数据分析几个方面来介绍大数据系统开发的实现技术,介绍大数据处理系统的一些常用的数据存储方案、常用的数据管理模式、常见的数据处理手段和数据分析方法。并简要说明大数据处理系统的部署需求及其运行环境。

19.3.1 数据存储

大数据处理系统以数据的分析处理作为核心功能,其首先必须具备强大的数据存储能力。数据存储的流程一般分为4个部分进行描述,分别是数据采集、数据清洗、数据转换和数据存储。当数据途经以上4个步骤之后,其规范性、完整性以及合理性会得到进一步的保证,后续大数据处理系统的数据分析功能才能够更好地完成。

1. 数据采集

在大数据处理系统中,数据采集都是一个非常重要的部分:主要负责数据的输入。大数据处理系统采集常用方法包括系统日志采集、利用 ETL 工具采集以及网络爬虫等。其中,日志文件是由数据源系统自动生成的记录文件,在 Web 服务器主要包括以下三种日志文件格式:公用日志文件格式、扩展日志格式和 IIS 日志格式;ETL 即数据抽取(Extract)、转换(Transform)、加载(Load)的过程;互联网爬虫则可以分为通用网络爬虫、聚焦网络爬虫、增量式网络爬虫和分布式网络爬虫。常用的大数据处理系统的数据采集主要分为以下三种采集模式。

1) 离线采集

在当前数据仓库的语境下，ETL 基本上就是数据采集的主要方式。在转换的过程中，需要针对具体的业务场景对数据进行治理。例如，进行非法数据监测与过滤、格式转换与数据规范化、数据替换、保证数据完整性等。

2) 实时采集

实时采集主要用在考虑流处理的业务场景。例如，用于记录数据源执行的各种操作活动；再如，网络监控的流量管理、金融应用的股票记账和 Web 服务器记录的用户访问行为。在流处理场景中，数据采集会成为 Kafka 的消费者，就像一个水坝一般将上游源源不断的数据拦截住，然后根据业务场景做对应的处理（如去重、去噪、中间计算等），之后再写入对应的数据存储中。这个过程类似传统的 ETL，但它是流式的处理方式，而非定时的批处理 Job。这些工具均采用分布式架构，能满足每秒数百兆字节（MB）的日志数据采集和传输需求。

3) 互联网采集

Scribe 是 Facebook 开发的数据（日志）收集系统，又被称为网页蜘蛛、网络机器人。它是一种按照一定的规则，自动抓取万维网信息的程序或者脚本，它支持图片、音频、视频等文件或附件的采集。爬虫除了网络中包含的内容之外，对于网络流量的采集可以使用 DPI 或 DFI 等带宽管理技术进行处理。

2. 数据清洗

数据清洗，是整个数据分析过程中不可缺少的一个环节，其结果质量直接关系到模型效果和最终结论。在实际操作中，数据清洗通常会占据分析过程的 50% ~ 80% 的时间。在大数据处理系统中，数据分析作为最重要的一个模板之一，需要提供强大的算力和支持以供数据处理的准确性和可靠性。

数据清洗的基本流程一共分为 5 个步骤，分别是数据分析、定义数据清洗的策略和规则、搜寻并确定错误实例、纠正发现的错误以及干净数据回流。

1) 数据分析

数据分析是数据清洗的前提和基础，通过人工检测或者计算机分析程序的方式对原始数据源的数据进行检测分析，从而得出原始数据源中存在的数据质量问题。

2) 定义数据清洗的策略和规则

根据数据分析出的数据源个数和数据源中的“脏”数据程度定义数据清洗策略和规则，并选择合适的数据清洗算法。

3) 搜寻并确定错误实例

搜寻并确定错误实例步骤包括自动检测属性错误和检测重复记录的算法。手工检测数据集中的属性错误需要花费大量的时间、精力以及物力，并且该过程本身很容易出错，所以需要使用高效的方法自动检测数据集中的属性错误，主要检测方法有基于统计的方法、聚类方法和关联规则方法。

检测重复记录的算法可以对两个数据集或者一个合并后的数据集进行检测，从而确定同一

个现实实体的重复记录，即匹配过程。检测重复记录的算法有基本的字段匹配算法、递归字段匹配算法等。

4) 纠正发现的错误

根据不同的“脏”数据存在形式的不同，执行相应的数据清洗和转换步骤解决原始数据源中存在的质量问题。需要注意的是，对原始数据源进行数据清洗时，应该将原始数据源备份，以防需要撤销清洗操作。

为了便于处理单数据源、多数据源以及单数据源与其他数据源合并的数据质量问题，一般需要在各个数据源上进行数据转换操作，具体如下。首先，从原始数据源的属性字段中抽取值（属性分离）。原始数据源的属性一般包含很多信息，这些信息有时需要细化成多个属性，便于后续清洗重复记录。其次，确认并改正数据，确认并改正输入和拼写的错误，然后尽可能地使该步骤自动化。若是基于字典查询拼写错误，则更利于发现拼写的错误。最后，进行标准化工作，标准化主要是为了便于记录实例匹配和合并，应该将属性值转换成统一格式。

5) 干净数据回流

当数据被清洗后，干净的数据替代原始数据源中的“脏”数据，这样可以提高信息系统的数据质量，还可避免将来再次抽取数据后进行重复的清洗工作。

3. 数据转换

将抽取出来的数据进行处理，并且将数据映射到仓库模型的维度（如时间、地点、产品等）和度量（如销售额、利润等）上。在这个过程中，需要进行数据清洗，如去除重复数据、数据类型转换等，以及数据规范化、合并、拆分等操作。

数据转换的主要目的是将数据转换为可用格式，数据转换其实就是 ETL 过程的一部分。ETL 从多个源中提取和转换数据是一个数据转换过程，并将其加载到数据仓库或其他目标系统中。在大数据处理系统中将数据转换的行为描述为 6 个步骤，分别是数据发现、数据映射、数据提取、代码生成、代码执行、数据审查。

1) 数据发现

在数据转换的第一步中，使用数据分析工具或手动分析脚本来分析数据。这有助于更好地理解数据的特征和结构，从而决定如何转换数据。

2) 数据映射

数据映射涉及定义如何映射、连接、聚合、修改或过滤每个字段以生成最终输出。它通常是在数据映射软件的帮助下进行的。数据映射通常是数据转换过程中最耗时、最昂贵的步骤。

3) 数据提取

在这一步中，数据是从原始来源中提取的。如上所述，来源可能有很大的差异，也可能包括结构化来源。

4) 代码生成

这一步涉及以 Python、R 或 SQL 等待语言生成可执行代码。可执行代码将根据定义的数据映射规则转换数据。

5) 代码执行

在此步骤中，生成的代码在数据上执行，以将其转换为所需格式。

6) 数据审查

在数据转换的最后一步中，检查输出数据是否符合转换要求。此步骤通常由数据的最终用户或业务用户执行。此步骤中发现的异常或错误将传达给数据分析师或开发人员。

4. 数据存储

数据存储是大数据处理系统的核心功能，只有提供安全稳定的数据存储能力，数据分析的结果才能得到有效的存储和收集。目前，大数据存储有两种方案可供选择：行存储和列存储。业界对两种存储方案有很多争执，关注的焦点是：谁能够更有效地处理海量数据，且兼顾安全、可靠、完整性。从目前发展情况看，关系数据库已经不适应这种巨大的存储量和计算要求，基本是淘汰出局。在已知的几种大数据处理软件中，Hadoop 的 HBase 采用列存储，MongoDB 是文档型的行存储，Lexst 是二进制型的行存储。

在列存储方式下，存储空间中的下一个对象就从同一条记录的下一个属性转变为下一条记录的同一属性。虽然这种旋转了 90 度的存储方式并没有减少数据量，但会带来以下好处：

(1) 大数据应用往往需要批量访问列数据（当用户主要关心同一属性的统计特性时），这时列存储方式的优势就会体现出来，列存储方式对属性的访问比行存储方式快很多。据有关报道，行存储方式的读取速度比行存储方式要快 50 ~ 100 倍。

(2) 有利于提高数据的压缩比，同类数据存储在一起有助于提高数据之间的相关性，从而有利于实施高效压缩算法（如行程压缩算法等）。

行存储以一行记录为单位，列存储以列数据集合为单位，或称列族（Column Family）。行存储的读写过程是一致的，都是从第一列开始，到最后一列结束。列存储的读取是列数据集中的一段或者全部数据，写入时，一行记录被拆分为多列，每列数据追加到对应列的末尾处。

行存储方式与列存储方式的对比如表 19-1 所示。

表 19-1 行存储方式与列存储方式对比表

比较对象	行存储方式	列存储方式
优点	写入效率高，保证数据完整性	读取过程没有冗余，适合数据定长的大数据计算
缺点	数据读取有冗余现象，影响计算速度	缺乏数据完整性保证，写入效率低
改进	优化存储格式，保证能够在内存快速删除冗余数据	多磁盘多线程并行读写（需要增加运营成本 and 修改软件）

从表 19-1 可以看出，行存储的写入是一次完成的。如果这种写入建立在操作系统的文件系统上，可以保证写入过程的成功或者失败，数据的完整性因此可以确定。列存储由于需要把一行记录拆分成单列保存，写入次数明显比行存储多，再加上磁头需要在盘片上移动和定位花费的时间，实际时间消耗会更大。所以，行存储在写入上占有很大的优势。

目前来说，MySQL 行存储的方式比较适合 OLTP 业务。列存储的方式比较适合 OLAP 业